The Structure-Preserving Doubling Algorithm

Solve a Riccati equation by joining optimal-control blocks

> Ordinary Riccati iteration adds one time step. SDA joins a block to a copy of itself, eliminates their shared boundary, and doubles the effective horizon: $1, 2, 4, 8, \ldots$. The acceleration is a consequence of exact composition.

1. The problem: how much does the future cost?

Consider the time-invariant, discrete-time linear-quadratic regulator (LQR):

$$x_{t+1} = Ax_t + Bu_t, \quad J = \frac{1}{2}\sum_{t=0}^{\infty}(x_t^T Q x_t + u_t^T R u_t). \tag{1}$$

Here $x_t \in \mathbb{R}^n$, $u_t \in \mathbb{R}^m$, $Q = Q^T \succeq 0$, and $R = R^T \succ 0$. The optimal infinite-horizon value is $V(x) = \frac{1}{2}x^T P x$. Bellman's principle gives the discrete-time algebraic Riccati equation (DARE):

$$P = Q + A^T P A - A^T P B\left(R + B^T P B\right)^{-1} B^T P A. \tag{2}$$

We assume (A, B) is stabilizable and $\left(Q^{\frac{1}{2}}, A\right)$ is detectable. These standard sufficient conditions ensure a unique positive-semidefinite stabilizing solution. "Stabilizing" means that the feedback

$$K = \left(R + B^T P B\right)^{-1} B^T P A, \quad u = -Kx \tag{3}$$

makes $A - BK$ Schur: all its eigenvalues have modulus less than one.

Set $G = BR^{-1}B^T$. Eliminating the control, or using the matrix inversion lemma, rewrites Equation 2 as

$$P = Q + A^T P (I + GP)^{-1} A. \tag{4}$$

Order matters: these are matrices, not scalars. In particular, $P(I + GP)^{-1} = (I + PG)^{-1}P$.

For a finite horizon of N stages and **zero terminal cost**, let P_N be its value matrix. Then $P_0 = 0$ and

$$P_{N+1} = Q + A^T P_N (I + GP_N)^{-1} A. \tag{5}$$

Thus ordinary iteration computes horizons $1, 2, 3, \ldots$. SDA will compute $P_1, P_2, P_4, P_8, \ldots$ without visiting every intermediate horizon.

2. From optimal-control conditions to doubling

We will derive the state–costate equations from the constrained finite-horizon LQR problem. The costates are unknown Lagrange multipliers, determined along with the states and inputs. No terminal price is prescribed, and no part of the terminal cost is discarded.

2.1. State the finite-horizon problem and its full Lagrangian

Fix x_0. Choose the inputs $u_0, \dots, u_{N-1}$ and states $x_1, \dots, x_N$ to minimize

$$J_N = \frac{1}{2}\sum_{t=0}^{N-1}(x_t^T Q x_t + u_t^T R u_t) + \frac{1}{2}x_N^T F x_N, \quad F = F^T \succeq 0, \tag{6}$$

subject to $x_{t+1} = Ax_t + Bu_t$ for $t = 0, \dots, N-1$. The last term is the **actual terminal cost**. Set $F = 0$ for the zero-terminal-cost problem whose value matrix is P_N in Section 1. A nonzero F lets us see explicitly where the terminal boundary condition comes from.

Introduce a multiplier λ_{t+1} for each dynamics constraint. With the sign convention $Ax_t + Bu_t - x_{t+1} = 0$, the **full** Lagrangian is

$$\mathcal{L} = J_N + \sum_{t=0}^{N-1}\lambda_{t+1}^T(Ax_t + Bu_t - x_{t+1}). \tag{7}$$

Every constraint residual appears in full, including $-x_{t+1}$. On feasible trajectories these residuals vanish, so $\mathcal{L} = J_N$.

Step 1: vary the input. Only the input cost and its own dynamics residual depend on u_t, so stationarity gives

$$\nabla_{u_t}\mathcal{L} = Ru_t + B^T\lambda_{t+1} = 0. \tag{8}$$

Step 2: vary an interior state. For $1 \le t \le N-1$, x_t appears in three places: its stage cost, the preceding residual through $-\lambda_t^T x_t$, and the following residual through $\lambda_{t+1}^T A x_t$. Therefore

$$\nabla_{x_t}\mathcal{L} = Qx_t - \lambda_t + A^T\lambda_{t+1} = 0, \tag{9}$$

or $\lambda_t = Qx_t + A^T\lambda_{t+1}$. This is the **costate recursion**.

Step 3: vary the terminal state. There is no stage cost or outgoing dynamics residual at $t = N$. The terms involving x_N are $\frac{1}{2}x_N^T F x_N - \lambda_N^T x_N$, giving

$$\nabla_{x_N}\mathcal{L} = Fx_N - \lambda_N = 0 \quad \Longrightarrow \quad \lambda_N = Fx_N. \tag{10}$$

Step 4: vary each multiplier. This recovers every dynamics constraint:

$$\nabla_{\lambda_{t+1}}\mathcal{L} = Ax_t + Bu_t - x_{t+1} = 0. \tag{11}$$

Together, these equations determine the solution. After eliminating the states, the objective is strictly convex in the input sequence because $R \succ 0$ and $Q, F \succeq 0$. Thus these first-order conditions are also sufficient for the unique optimal input sequence.

2.2. Interpret the costate and retain both boundary conditions

The state equation propagates forward, but the costate recursion propagates backward from Equation 10. The boundary conditions are

$$x_0 = \text{given}, \quad \lambda_N = F x_N. \tag{12}$$

For $F = 0$, the terminal condition is $\lambda_N = 0$; for $F \neq 0$, neither x_N nor λ_N is generally known beforehand. They must be solved consistently with all the other equations.

Let $V_t(x_t)$ be the optimal remaining cost from time t. Bellman's principle and stationarity imply $\lambda_t = \nabla V_t(x_t)$ along the optimum. In particular, the initial value sensitivity is

$$\lambda_0 := \nabla V_0(x_0) = Q x_0 + A^T \lambda_1. \tag{13}$$

There is no multiplier for a dynamics constraint ending at x_0 in Equation 7: x_0 is fixed. Here λ_0 is defined as a sensitivity, not obtained by imposing $\nabla_{x_0} \mathcal{L} = 0$. Differentiating the optimized Lagrangian with respect to that initial-state parameter gives the expression above. The costate is therefore a gradient of optimal cost, not a second physical state.

> A partial derivative of the Lagrangian treats states, inputs, and multipliers as independent variables. It does **not** require that the dynamics remain satisfied during that variation. Feasibility is imposed by the multiplier equations, and all stationarity and boundary conditions must hold together. If instead x_1 is fixed while enforcing $x_1 = Ax_0 + Bu_0$, the feasible inputs are restricted; when $B = 1$, $u_0 = x_1 - Ax_0$ is already determined.

2.3. Eliminate the input, not the terminal condition

From Equation 8, $u_t = -R^{-1} B^T \lambda_{t+1}$. Substitution into the dynamics gives, with $G = BR^{-1}B^T$,

$$x_{t+1} = Ax_t - G\lambda_{t+1}, \quad \lambda_t = Qx_t + A^T \lambda_{t+1}. \tag{14}$$

These are the discrete-time state–costate optimality equations after input elimination: the discrete-time Pontryagin conditions for LQR. For a single step, the terminal condition is $\lambda_1 = Fx_1$. Substitute it into the first equation:

$$(I + GF)x_1 = Ax_0, \quad x_1 = (I + GF)^{-1} A x_0. \tag{15}$$

The second then gives

$$\lambda_0 = [Q + A^T F (I + GF)^{-1} A] x_0. \tag{16}$$

Thus the one-step value matrix is $Q + A^T F (I + GF)^{-1} A$: the Riccati update with downstream value matrix F. Both the dynamics constraint and the actual terminal cost have been retained.

2.4. Join two steps and eliminate their shared boundary

The first step and the second step obey, respectively,

$$x_1 = Ax_0 - G\lambda_1, \quad \lambda_0 = Qx_0 + A^T\lambda_1, \tag{17}$$

$$x_2 = Ax_1 - G\lambda_2, \quad \lambda_1 = Qx_1 + A^T\lambda_2. \tag{18}$$

These are consecutive equations from the **same** optimal-control problem, with terminal condition $\lambda_2 = Fx_2$. The interior multiplier λ_1 is shared because it belongs to the constraint ending at x_1; it is also the costate at the beginning of the second step. We will eliminate the interior pair (x_1, λ_1), while leaving the terminal condition intact.

Step 1: eliminate the interior costate. Substitute the second step's λ_1 into the first step's state equation:

$$x_1 = Ax_0 - G(Qx_1 + A^T\lambda_2) = Ax_0 - GQx_1 - GA^T\lambda_2. \tag{19}$$

Set $W = I + GQ$ and collect the x_1 terms:

$$Wx_1 = Ax_0 - GA^T\lambda_2, \quad x_1 = W^{-1}Ax_0 - W^{-1}GA^T\lambda_2. \tag{20}$$

Step 2: substitute into the terminal state equation.

$$x_2 = Ax_1 - G\lambda_2 = (AW^{-1}A)x_0 - (G + AW^{-1}GA^T)\lambda_2. \tag{21}$$

Thus define $E_2 = AW^{-1}A$ and $G_2 = G + AW^{-1}GA^T$.

Step 3: substitute into the initial costate equation. First,

$$\lambda_0 = Qx_0 + A^T(Qx_1 + A^T\lambda_2). \tag{22}$$

After replacing x_1 and collecting terms,

$$\lambda_0 = (Q + A^TQW^{-1}A)x_0 + A^T(I - QW^{-1}G)A^T\lambda_2. \tag{23}$$

Define $H_2 = Q + A^TQW^{-1}A$. The terminal-costate coefficient simplifies via

$$I - QW^{-1}G = (I + QG)^{-1}, \quad A^T(I + QG)^{-1}A^T = E_2^T. \tag{24}$$

The inverse identity is proved in Section 2.7. The two-step block has the **same shape** as one step:

$$x_2 = E_2x_0 - G_2\lambda_2, \quad \lambda_0 = H_2x_0 + E_2^T\lambda_2. \tag{25}$$

Close the boundary-value problem. We have not solved the original optimization until we also impose $\lambda_2 = Fx_2$. Thus

$$x_2 = (I + G_2F)^{-1}E_2x_0, \tag{26}$$

$$\lambda_0 = \left[H_2 + E_2^TF(I + G_2F)^{-1}E_2\right]x_0. \tag{27}$$

For $F = 0$, this reduces to $\lambda_0 = H_2x_0$, so $H_2 = P_2$.

2.5. See the two-step elimination in numbers

Use the running scalar example $A = 1.1$, $B = Q = R = 1$, so $G = 1$. At the shared boundary,

$$x_1 = 1.1x_0 - \lambda_1, \quad \lambda_1 = x_1 + 1.1\lambda_2. \tag{28}$$

Substitute the second equation into the first:

$$x_1 = 1.1x_0 - x_1 - 1.1\lambda_2 \quad \Longrightarrow \quad x_1 = 0.55x_0 - 0.55\lambda_2. \tag{29}$$

Recover the terminal state:

$$x_2 = 1.1x_1 - \lambda_2 = 0.605x_0 - 1.605\lambda_2. \tag{30}$$

Recover the initial costate, using $\lambda_1 = x_1 + 1.1\lambda_2$:

$$\lambda_0 = x_0 + 1.1\lambda_1 = x_0 + 1.1(x_1 + 1.1\lambda_2) = 1.605x_0 + 0.605\lambda_2. \tag{31}$$

So the whole two-step problem is summarized by

$$E_2 = 0.605, \quad G_2 = 1.605, \quad H_2 = 1.605. \tag{32}$$

The interior variables x_1, λ_1 have disappeared. If there is no terminal cost, $\lambda_2 = 0$; then $\lambda_0 = 1.605x_0$ and the optimized two-step value is $V_2(x_0) = \frac{1}{2} \cdot 1.605x_0^2$. This is why H_2 is the finite-horizon Riccati solution, not just an arbitrary coefficient.

For a nonzero terminal weight, take $x_0 = 1$ and $F = 1$. The terminal condition is $\lambda_2 = x_2$, not an independently chosen multiplier. The condensed equations give

$$x_2 = 0.605 - 1.605x_2 \quad \Longrightarrow \quad x_2 = \frac{0.605}{2.605} \approx 0.232246, \tag{33}$$

$$\lambda_0 = 1.605 + 0.605x_2 \approx 1.745509. \tag{34}$$

This explicitly closes the two-step optimal-control problem with its quadratic terminal cost.

2.6. Only now introduce the general block

One optimized step has a linear boundary relation. We have shown that joining two steps and eliminating the shared boundary preserves its form. Repeating the same operation joins two two-step blocks into a four-step block, then two four-step blocks into an eight-step block. This motivates the notation

$$x_N = E_N x_0 - G_N \lambda_N, \quad \lambda_0 = H_N x_0 + E_N^T \lambda_N. \tag{35}$$

These are **boundary relations**, not an uncontrolled forward state transition: they condense the state–costate equations by eliminating interior variables. They must still be combined with the original endpoint conditions $x_0 =$ given and $\lambda_N = Fx_N$. Leaving λ_N symbolic during elimination does not make it a prescribed terminal price.

- E_N transmits the initial state across the optimized block.
- G_N couples the terminal costate to the terminal state.
- H_N is the initial value curvature when $\lambda_N = 0$.

For zero terminal cost, $\lambda_N = 0$ and $\lambda_0 = H_N x_0$. But the zero-terminal-cost optimal value is $\frac{1}{2}x_0^T P_N x_0$, whose gradient is $P_N x_0$. Therefore $H_N = P_N$. The one-step coefficients are $(E_1, G_1, H_1) = (A, BR^{-1}B^T, Q)$, directly from Equation 14 and Equation 13.

> A block is a condensed system of optimality equations, not a replacement objective. SDA eliminates interior states and costates while retaining the original endpoint conditions. For zero terminal cost, $\lambda_N = 0$ and the block coefficient H_N is the finite-horizon value matrix P_N.

2.7. Derive the three doubling updates without skipping the algebra

For a current N-step block, temporarily write its coefficients as (E, G, H); here G is the **current block coefficient**, not necessarily the original $BR^{-1}B^T$. Two identical copies of Equation 35 give

$$x_N = Ex_0 - G\lambda_N, \quad \lambda_0 = Hx_0 + E^T\lambda_N, \tag{36}$$

$$x_{2N} = Ex_N - G\lambda_{2N}, \quad \lambda_N = Hx_N + E^T\lambda_{2N}. \tag{37}$$

Step 1: eliminate the shared costate, just as for two steps.

$$x_N = Ex_0 - G(Hx_N + E^T\lambda_{2N}). \tag{38}$$

With $W = I + GH$, collect terms:

$$(I + GH)x_N = Ex_0 - GE^T\lambda_{2N}. \tag{39}$$

Thus $x_N = W^{-1}Ex_0 - W^{-1}GE^T\lambda_{2N}$.

Step 2: collect the two coefficients in the terminal state.

$$x_{2N} = Ex_N - G\lambda_{2N} = (EW^{-1}E)x_0 - (G + EW^{-1}GE^T)\lambda_{2N}. \tag{40}$$

Reading off the coefficients gives $E_+ = EW^{-1}E$ and $G_+ = G + EW^{-1}GE^T$.

Step 3: collect the two coefficients in the initial costate.

$$\lambda_0 = Hx_0 + E^T(Hx_N + E^T\lambda_{2N}) \tag{41}$$

$$\lambda_0 = (H + E^THW^{-1}E)x_0 + E^T(I - HW^{-1}G)E^T\lambda_{2N}. \tag{42}$$

Hence $H_+ = H + E^THW^{-1}E$. To identify the remaining coefficient as E_+^T, we need two identities, not commutativity.

Step 4: justify the inverse identities. First,

$$(I + GH)G = G(I + HG) \quad \Longrightarrow \quad W^{-1}G = G(I + HG)^{-1}. \tag{43}$$

This follows by multiplying on the left by W^{-1} and on the right by $(I + HG)^{-1}$. Next, insert this identity and use $HG = (I + HG) - I$:

$$I - HW^{-1}G = I - HG(I + HG)^{-1} = (I + HG)^{-1}. \tag{44}$$

Since G, H are symmetric, $W^T = I + HG$, so the terminal-costate coefficient in λ_0 is $E^TW^{-T}E^T = \left(EW^{-1}E\right)^T = E_+^T$. The new block again has form Equation 35, with the standard SDA updates

$$E_+ = E(I + GH)^{-1}E, \quad G_+ = G + EG(I + HG)^{-1}E^T, \tag{45}$$

$$H_+ = H + E^TH(I + GH)^{-1}E. \tag{46}$$

> Initialize $(E_0, G_0, H_0) = (A, BR^{-1}B^T, Q)$ and apply these updates simultaneously. We now switch notation: the subscript k indexes **doublings**, not horizon length. Iteration 0 represents one step; iteration k represents 2^k steps, so $H_k = P_{2^k}$. The zero terminal condition is imposed only at the end of the **whole** horizon, not at the internal boundary between its two halves.

3. What structure survives the doubling?

3.1. Symmetry, positivity, and a monotone value

Suppose G, H are symmetric positive semidefinite. Although $I + GH$ need not be symmetric, it is nonsingular: GH has nonnegative real eigenvalues (its nonzero eigenvalues coincide with those of $H^{\frac{1}{2}}GH^{\frac{1}{2}}$). Moreover,

$$H(I + GH)^{-1} = H^{\frac{1}{2}}\left(I + H^{\frac{1}{2}}GH^{\frac{1}{2}}\right)^{-1}H^{\frac{1}{2}} \succeq 0. \tag{47}$$

This identity remains valid for singular H; no inverse of H is needed. The analogous identity holds for $G(I + HG)^{-1}$. Therefore Equation 46 preserves symmetry and positivity, and

$$H_+ \succeq H, \quad G_+ \succeq G. \tag{48}$$

The first inequality has an optimal-control interpretation: adding nonnegative stage costs cannot reduce the zero-terminal-cost optimal value. Under our LQR assumptions, H_k increases to the stabilizing P.

3.2. The symplectic connection, without assuming invertible dynamics

Let $J = \left(\begin{smallmatrix} 0 & I \\ -I & 0 \end{smallmatrix}\right)$, and write the boundary relation as a matrix pencil:

$$M\begin{pmatrix} x_0 \\ \lambda_0 \end{pmatrix} = L\begin{pmatrix} x_N \\ \lambda_N \end{pmatrix}, \quad M = \begin{pmatrix} E & 0 \\ -H & I \end{pmatrix}, \quad L = \begin{pmatrix} I & G \\ 0 & E^T \end{pmatrix}. \tag{49}$$

For symmetric G, H, direct multiplication gives

$$MJM^T = LJL^T = \begin{pmatrix} 0 & E \\ -E^T & 0 \end{pmatrix}. \tag{50}$$

This is the symplectic matrix-pair structure that the doubling retains in exact arithmetic. If E is invertible, $S = L^{-1}M$ is an ordinary symplectic state–costate transition: $S^T JS = J$. The pair formulation and SDA still make sense when E is singular; forming that transition is unnecessary.

3.3. Why convergence accelerates

The discrete Riccati map has derivative at its stabilizing solution

$$D\mathcal{F}_{P[\Delta]} = A_c^T \Delta A_c, \quad A_c = (I + GP)^{-1}A = A - BK. \tag{51}$$

Since A_c is Schur, ordinary iteration converges geometrically asymptotically. For any fixed r with $\rho(A_c) < r < 1$, sufficiently large horizons admit a bound of the form $\|P - P_N\| \le Cr^{2N}$. Sampling at $N = 2^k$ gives

$$\|P - H_k\| \le Cr^{2\cdot 2^k}. \tag{52}$$

This explains the quadratic-type convergence in the doubling index; a bound alone is not an exact error recurrence. The formal SDA convergence analysis is given in Section 2 of [1]. Modes close to the unit circle can delay the fast asymptotic regime. Structure preservation is not a guarantee of perfect conditioning or of exact symmetry in floating-point arithmetic.

4. A scalar example you can check by hand

Take $A = 1.1$, $B = Q = R = 1$. Then $G = 1$ and

$$p = 1 + \frac{1.21p}{1+p}, \quad p^2 - 1.21p - 1 = 0. \tag{53}$$

The positive solution is

$$p_\star = \frac{1.21 + \sqrt{1.21^2 + 4}}{2} = 1.773770721741\ldots \tag{54}$$

Its closed-loop multiplier is $\frac{1.1}{1+p_\star} \approx 0.39657$, inside the unit circle. The negative root is not the stabilizing solution.

Write the scalar SDA variables as (e, g, h). Each iteration uses

$$d = 1 + gh, \quad e_+ = \frac{e^2}{d}, \quad g_+ = g + e^2\frac{g}{d}, \quad h_+ = h + e^2\frac{h}{d}. \tag{55}$$

Starting at $(1.1, 1, 1)$, the first update is $(e_+, g_+, h_+) = (0.605, 1.605, 1.605)$. The next h is $1.605 + 0.605^2 \cdot \frac{1.605}{1+1.605^2} \approx 1.769280206374$.

Iteration	Horizon	H_k	Absolute error
0	1	1.000000000000	7.738×10^{-1}
1	2	1.605000000000	1.688×10^{-1}
2	4	1.769280206374	4.491×10^{-3}
3	8	1.773767969365	2.752×10^{-6}
4	16	1.773770721740	1.030×10^{-12}
5	32	1.773770721741	2.220×10^{-16}

Table 1: Double-precision Octave results. After five doublings the error reaches the floating-point floor; further squaring cannot add reliable digits.

A visual measure: correct decimal digits. The bar length below represents $-\log_{10}(|p_\star - H_k|)$, capped at 16. Each iteration eventually gains roughly twice as many digits, until rounding dominates.



The speedup is in **iteration count**, not in matrix size. In the dense case, each doubling still requires $O(n^3)$ arithmetic and $O(n^2)$ storage.

5. Implementation: solve systems and check the original problem

The following Octave/MATLAB loop implements Equation 46. The transpose solve uses $(I + GH)^T = I + HG$; it does not assume $GH = HG$.

```
G = B * (R \ B'); H = Q; E = A;
G0 = G; n = size(A, 1); converged = false;
tol = 1e-12; maxit = 50;
for k = 1:maxit
    W = eye(n) + G*H;
    U = W \ E;
    V = W' \ E';
    En = E*U;
    Gn = G + E*G*V;
    Hn = H + E'*H*U;
    % All right-hand sides above use OLD E, G, H.
    E = En; G = Gn; H = Hn;
    % Remove only roundoff-level antisymmetric components.
    G = (G+G')/2; H = (H+H')/2;
    T = A'*H*((eye(n)+G0*H) \ A);
    residual = norm(Q + T - H, 'fro');
    scale = max(1, norm(Q,'fro') + norm(T,'fro') ...
                   + norm(H,'fro'));
    K = (R+B'*H*B) \ (B'*H*A);
    stable = max(abs(eig(A-B*K))) < 1;
    if residual/scale < tol && stable
        converged = true; break;
    end
end
if ~converged, error('SDA failed validation'); end
P = H;
```

Why these checks? A small increment can mean convergence or stagnation. The residual tests the original DARE, while the stability check distinguishes the stabilizing solution from other algebraic solutions. A small residual is not a forward-error certificate for an ill-conditioned problem; near-unit-circle eigenvalues also need a more careful stability assessment than this teaching loop. Symmetrization removes rounding noise, not an incorrect matrix ordering.

A noncommuting check. With

$$A = \begin{pmatrix} 1.1 & 0.3 \\ 0 & 0.95 \end{pmatrix}, \quad B = \begin{pmatrix} 0.2 \\ 1 \end{pmatrix}, \quad Q = \begin{pmatrix} 2 & 0.4 \\ 0.4 & 1 \end{pmatrix}, \quad R = 0.7, \tag{56}$$

eight raw SDA updates (without symmetrization) gave a Frobenius difference of 2.2×10^{-15} from Octave's `dare` solution and an absolute compact-form residual of 6.3×10^{-16}. Unlike the scalar example, this checks matrix order.

Scope. These formulas are for the standard positive-semidefinite DARE. Continuous-time, indefinite, descriptor, and large sparse problems need suitable transformations or specialized variants, not blind reuse of this initialization.

Further reading

[1] E. K.-W. Chu, H.-Y. Fan, W.-W. Lin, and C.-S. Wang, "Structure-preserving algorithms for periodic discrete-time algebraic Riccati equations," *International Journal of Control*, vol. 77, no. 8, pp. 767–788, 2004, doi: 10.1080/00207170410001714988.